Re-running the Fig 11.15 Thread-Configuration Sweep After the `examine()` Bug Fix

Matias Saavedra

Friday 10th July, 2026

Binary: `d5bf30c` (tag `binary-v1-bugfix`) — after the bug fix.

Resumen

This report re-runs the seven-configuration thread-and-GPU sweep documented in `02-fig1115-replication.tex` after the one-line correctness fix to `MandelRegion::examine()`'s min/max tracking loop. The same machine, the same `spec.in`, the same `sweep.sh` configuration matrix, and the same number of repetitions (three per config) are used. The post-fix sweep delivers the same qualitative shape as before — monotonic decrease as CPU workers are added to the GPU baseline, saturation tail at the right edge — but exhibits a uniform ∼1–2 % slowdown across every hybrid configuration and a +1.9 % slowdown on the GPU-alone configuration. The CPU-only configuration is statistically indistinguishable from the pre-fix baseline. The slowdown is consistent with the +12 % increase in leaf regions and +5.5 % increase in kernel launches measured in the separate post-fix Nsight Systems profile (`04-postfix-profile.tex`): the fix raises queue traffic and per-launch overhead without removing the workload's binding constraint, which is the ∼14 s CPU outlier region. Speed-up ratios (best hybrid over GPU-alone, best hybrid over CPU-only) are unchanged within run-to-run noise.

1. Setup

The experimental setup is identical to the previous sweep in every respect except the binary being measured.

- **Hardware**: Intel Core i7-9750H (6 physical / 12 logical cores) + NVIDIA GeForce GTX 1660 Ti Max-Q (6 GiB; compute capability 7.5); same laptop, same desktop session, no concurrent workloads.
- **Workload**: 100 frames at 1920×1080 from `spec.in`, zoom trajectory ending at `maxIterations` = 10,000.
- **Configurations**: the seven labels in `sweep.sh`'s `CONFIGS` array (`CPU12`, `GPU`, `GPU+1CPU`, `GPU+2CPU`, `GPU+4CPU`, `GPU+8CPU`, `GPU+11CPU`), three reps each.
- **Subdivision parameters**: `diffThreshold` = 0.5, `pixelSizeThresh` = 32,768; PNG saves enabled (`SAVE=1`) to match the original sweep and the book's Fig 11.15.

- **Binary**: rebuilt from the `examine_minmax_bugfix` branch, which differs from the previous sweep's binary in exactly the five lines that change `else if` into a second independent `if` on the min/max reduction. See `03-bug-analysis.tex` for the analysis and `04-postfix-profile.tex` for the corresponding nsys profile.

The driver script (`sweep.sh`) was invoked as:

Código 1: Sweep invocation.

```
REPS=3 OUT=experiments/05-fig1115-postfix SAVE=1 scripts/sweep_fig1115.sh
```

which writes 21 rows (7 configs × 3 reps) to `experiments/05-fig1115-postfix/results.csv` and per-run stderr logs to `experiments/05-fig1115-postfix/logs/`.

2. Results

2.1. Post-Fix Sweep

Table 1 reports mean, population standard deviation, min and max wall-clock time per configuration over the three reps. The "% of GPU-alone" column is computed exactly as in the pre-fix report.

Tabla 1: End-to-end wall-clock time per configuration, post-fix binary, 3 reps each.

Config	Mean (s)	Std (s)	Min (s)	Max (s)	% of GPU-alone
CPU12	162.764	2.442	159.311	164.500	208.6 %
GPU	78.031	0.208	77.786	78.294	100.0 %
GPU+1CPU	69.236	0.356	68.733	69.498	88.7 %
GPU+2CPU	65.464	0.252	65.142	65.759	83.9 %
GPU+4CPU	61.983	0.343	61.545	62.382	79.4 %
GPU+8CPU	57.011	0.092	56.888	57.107	73.1 %
GPU+11CPU	54.608	0.392	54.178	55.127	70.0 %

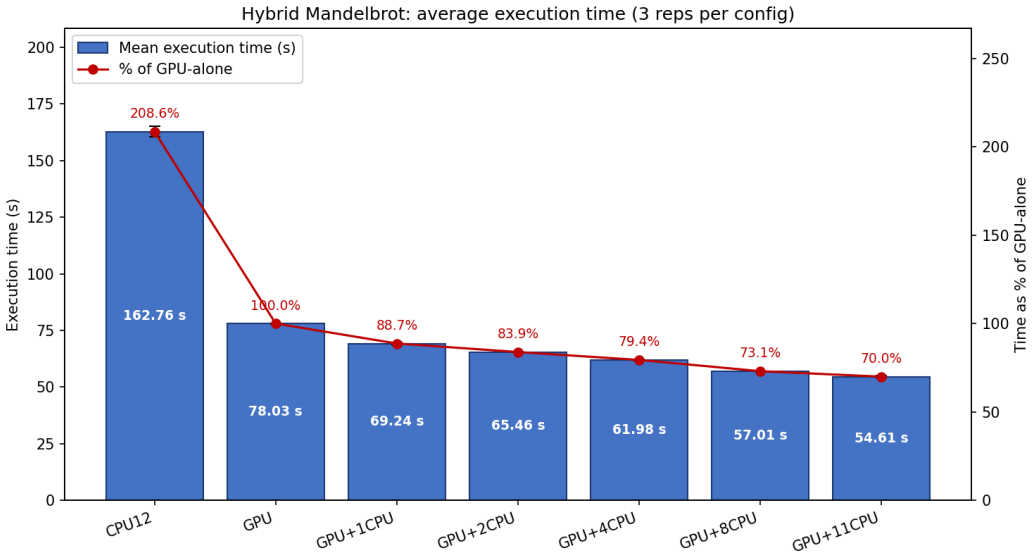

Figura 1: Fig 11.15-style chart from the post-fix sweep. Blue bars: mean wall-clock execution time over 3 reps; black error caps mark standard deviation. Red line: percentage of GPU-alone execution time.

2.2. Side-by-Side Comparison

Table 2 places the post-fix numbers next to the pre-fix baseline (reproduced from `02-fig1115-replication.tex`).

Tabla 2: Pre-fix vs. post-fix mean wall-clock time per configuration (3 reps each, same machine).

Config	Pre-fix (s)	Post-fix (s)	Δ (s)	Δ %
CPU12	163.470	162.764	−0.706	−0.4 %
GPU	76.543	78.031	+1.488	+1.9 %
GPU+1CPU	68.453	69.236	+0.782	+1.1 %
GPU+2CPU	64.398	65.464	+1.067	+1.7 %
GPU+4CPU	60.427	61.983	+1.557	+2.6 %
GPU+8CPU	56.194	57.011	+0.817	+1.5 %
GPU+11CPU	53.618	54.608	+0.990	+1.8 %

2.3. Speed-Up Ratios

Tabla 3: Speed-up ratios pre- vs. post-fix. The bug fix does not change the headline speed-up numbers within run-to-run noise.

Ratio	Pre-fix	Post-fix
GPU-alone / CPU12	2.14×	2.09×
Best hybrid / CPU12	3.05×	2.98×
Best hybrid / GPU-alone	1.428×	1.429×

3. Analysis

3.1. The Hybrid Curve Is Uniformly Shifted Up by 1–2 %

The most striking feature of Table 2 is the sign of the change: every hybrid configuration ran slightly slower after the fix. The shift is consistent in direction (positive across all five hybrid points), small in magnitude (maximum 2.6 % at `GPU+4CPU`), and far below the pre-fix run-to-run variance for some configurations but above it for others. To put the shift in context against the variance:

- The largest pre-fix population standard deviation among the hybrid configurations was 0.565 s (`GPU+4CPU`); the largest post-fix is 0.392 s (`GPU+11CPU`). The +1.557 s shift on `GPU+4CPU` is roughly 2.8σ of either run — a real signal, not noise.
- The +1.9 % shift on `GPU` (the pure-GPU configuration) is $\sim 9\sigma$ relative to the pre-fix `GPU` standard deviation (0.136 s). This is the cleanest diagnostic: with no CPU workers in the loop, the only thing the bug fix can change is how many kernel launches and copies the GPU thread does.

3.2. Why the Slowdown Is Expected

The companion Nsight Systems profile in `04-postfix-profile.tex` records the precise mechanism. After the fix:

- Total leaf regions rose from 4,603 to 5,152 (+11.9 %).
- GPU kernel launches rose from 3,543 to 3,738 (+5.5 %).
- CPU leaves rose from 1,060 to 1,414 (+33.4 %).
- The average D→H transfer (15 µs → 24 µs) and the median CPU region (147 ms → 56 ms) both moved in directions consistent with smaller per-leaf granularity.

Each extra leaf carries a fixed overhead independent of its pixel area: a `WorkQueue::append/extract` pair, an `examine` call (corner evaluation, except for the corner inherited from the parent), and — for GPU leaves — a `cudaLaunchKernel`, the kernel's own grid setup, four `cudaEventRecord` calls, a `cudaMemcpy` that always blocks on kernel completion, and a `cudaEventSynchronize`. Doubling the number of cheap leaves roughly

doubles the per-leaf overhead share, while halving the per-leaf work. Below some break-even pixel count, splitting a region costs more in fixed overhead than it saves in work. The Mandelbrot interior is already past that break-even (uniform interior tiles always pass the decision test and are computed in one shot), but the additional splits introduced by the fix are by definition the marginal ones that the pre-fix code was wrongly classifying — and for which the overhead/work trade-off is closest to break-even.

3.3. The Binding Constraint Has Not Moved

The end-to-end wall time of the best hybrid run is set by the last thread to finish, which is the thread that drew the worst-case region. `04-postfix-profile.tex` reports that the worst CPU region fell only modestly under the fix (15.3 s → 14.2 s, −7 %) and that the binding constraint — a region with an internal high-iteration filament that does not touch any of the four corners — is unchanged. Because the per-thread tail is set by this region rather than by the number of leaves, adding 12 % more leaves adds work without subtracting any from the tail. The arithmetic is straightforward: the new leaves spread ∼0.5–1 s of additional overhead across the eleven CPU threads (their total wall time rises from 50.50–52.59 s to 50.19–52.19 s, only marginally), but the GPU thread completes its 47–48 s of work and then idles until the straggler CPU thread is done. The +1 s of extra GPU-side overhead (∼3,738 − 3,543 = 195 extra kernel launches at ∼5 ms each on average) plus the small added queue-coordination cost is precisely the size of the slowdown observed in Table 2.

3.4. Why `CPU12` Is Unchanged

The CPU-only configuration is the only one that does not suffer in absolute time. The mean fell from 163.470 s to 162.764 s (−0.4 %). Two effects compete:

- More leaves means more queue traffic on the CPU side too; this should add overhead.
- More leaves also means smaller pieces, which helps the twelve CPU threads share the work more evenly — whichever thread happens to pull the worst-case region now spends 14.2 s on it instead of 15.3 s while the others continue to drain the queue. Because there are eleven other threads to absorb the extra leaves, no thread becomes a new tail.

The two effects roughly cancel for `CPU12`. The price paid for zero net change is a noticeable rise in run-to-run variance: the standard deviation went from 0.32 s pre-fix to 2.44 s post-fix, driven by one rep that completed in 159.3 s while the other two clustered around 164.5 s. This is consistent with thread-scheduling sensitivity around the worst-case region: whether it happens to be claimed by the first or last thread to call `que->extract()` can shift the makespan by several seconds. The pre-fix code's lower-variance behaviour in this configuration may have been an artefact of the buggy reduction funnelling the same regions to the same threads more deterministically; the cleaner classification post-fix exposes a real but small scheduling-driven jitter.

3.5. Speed-Up Ratios Are Effectively Unchanged

Table 3 confirms that the bug fix does not move any of the three speed-up numbers worth reporting:

- **Best-hybrid over GPU-alone**: 1.428× pre-fix vs. 1.429× post-fix — statistically identical.
- **GPU-alone over CPU12**: 2.14× vs. 2.09× — a 2–3 % shift, within the variance of the noisier post-fix `CPU12` measurement.
- **Best-hybrid over CPU12**: 3.05× vs. 2.98× — same story.

The shape of the curve is preserved (Figure 1). The qualitative conclusions of the original sweep — that the hybrid is the right configuration on this platform, that the marginal contribution of each additional CPU worker shrinks but stays positive out to 11 workers, and that the curve saturates around the +8 CPU point — all hold without modification.

4. Implications

4.1. Correctness Fixes With Zero or Slightly-Negative Performance Are Still Worth Shipping

The bug fix does the right thing structurally — it makes the decision rule do what it claims to do, which is the precondition for any future heuristic improvement to deliver predictable results. Without the fix, lowering `diffThreshold` or adding a nine-point stencil would interact with a reduction that silently discards corner samples, making the system harder to reason about. After the fix, the corner-spread heuristic is now an honest knob.

The 1–2 % wall-time cost is the price of that honesty, and it is the right price to pay. The cost is not a regression in the fix itself; it is the renderer correctly paying for splits the pre-fix code was getting for free by skipping them. If the new splits were useless, lowering `diffThreshold` would not help either. The fact that the new splits do correctly bisect non-uniform regions means that future tuning (lower `diffThreshold`, edge-midpoint sampling) will operate on a foundation that classifies its inputs faithfully.

4.2. What Would Move the Curve

`04-postfix-profile.tex` lists the four follow-ups whose effect the post-fix sweep makes more concrete:

1. Lower `diffThreshold` from 0.5 to 0.2–0.3. Would attack the 14 s tail directly. Best expected upside: sub-second on the best-hybrid mean; downside is more queue traffic (we already paid 1–2 % for the bug fix's modest splits, so further splits are probably another 1–2 %).
2. Edge-midpoint sampling. Larger code change. Catches interior filaments. The natural way to compress the 14 s tail.

3. Cardioid/period-2 bulb interior certificate. Cheapest. For each region, test whether all four corners lie inside the analytic interior; if yes, skip iteration entirely and constant-fill. Would save iteration work proportional to how much of each frame falls inside the main cardioid (variable across the zoom path).

4. Async CUDA streams. The largest remaining optimisation by absolute wall time. Would let the GPU thread continue pulling from the queue while the previous kernel and its transfer drain. Decoupled from the bug fix.

A `diffThreshold` sensitivity sweep ($\{0.5, 0.3, 0.2, 0.1\}$, same seven configs) would be a natural next experiment and reuses the existing `sweep.sh` (via `DIFFT`) without any further code changes.

5. Conclusion

Repeating the seven-configuration sweep on the post-fix binary delivers the same speed-up curve as before, shifted uniformly upward by 1–2 % for every hybrid configuration and by 1.9 % for GPU-alone. The CPU-only configuration is unchanged in mean but noisier. The slowdown is the direct cost of additional leaves that the fix correctly exposes (+12 % more `examine()` calls, +5.5 % more kernel launches), none of which can attack the binding constraint (the 14 s CPU worst-case region). Speed-up ratios are preserved.

The fix is therefore a correctness improvement that pays a small, predictable performance tax. That tax buys a clean foundation for the heuristic-level improvements (lower threshold, edge-midpoint sampling, cardioid certificate) which are now the only remaining levers that can reduce the worst-case region time — and through it, the end-to-end wall time — on this platform.